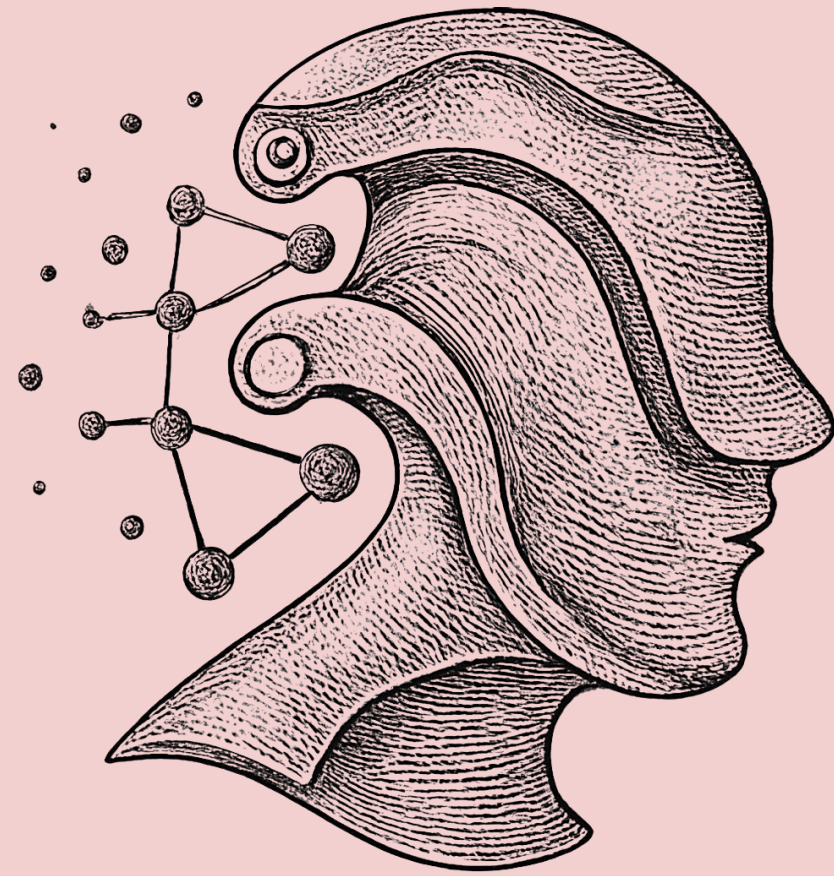




DeepFakeDetect

A Robust ResNet-50
System for Real-Time Fake Image Detection and Mobile APP Deployment

TEAM:

- Kuladeep (23bds021)
- Harshith (23bds058)
- Sadan (23bds048)
- Bharath (23bds012)
- Guru Sai Harsha (23bds005)
- Sudharshana (23bds059)

The Problem & Our Objective

The Threat:

The rapid progress of generative models (like GANs and diffusion) has made hyper-realistic forged images widely available. This threatens trust, privacy, and safety, making automatic detection critical for content moderation and forensics.

The "In-the-Wild" Challenge:

Modern fakes minimize the low-level artifacts older detectors relied on. & Images "in the wild" suffer from compression, resizing, and noise from being processed by social media platforms.

Our Objective:

To build a practical, end-to-end system that is robust to these "in-the-wild" degradations.

The DeepFake-Detect Pipeline

Model:

A fine-tuned ResNet-50 backbone for powerful feature extraction.

Classifier:

A custom regularized classifier head to replace the original, reducing overfitting

Deployment (Backend):

A FastAPI inference service for high-speed, scalable deployment.

Deployment (Client):

A React Native mobile client for a practical, real-world interface.

Dataset & Preprocessing

Source:

- Kaggle Deepfake Image Detection dataset.
<https://www.kaggle.com/datasets/saurabhbagchi/deepfake-image-detection/data>

Data Splits:

- Training: 882 images
- Validation: 47 images
- Testing (Held-out): 49 images (27 fake, 22 real)

Preprocessing:

- All images resized to $224 * 224$.
- Normalized with ImageNet mean and standard deviation²⁶.

Augmentations for Robustness:

- Standard: RandomResizedCrop, RandomHorizontalFlip.
- Real-World Simulation:
- ColorJitter (brightness, contrast, saturation)
- Gaussian Blur (to simulate focus/motion blur)
- Random JPEG compression (to model social-media re-encoding)

Model Architecture

Performance on Held-Out Test Set (49 images):

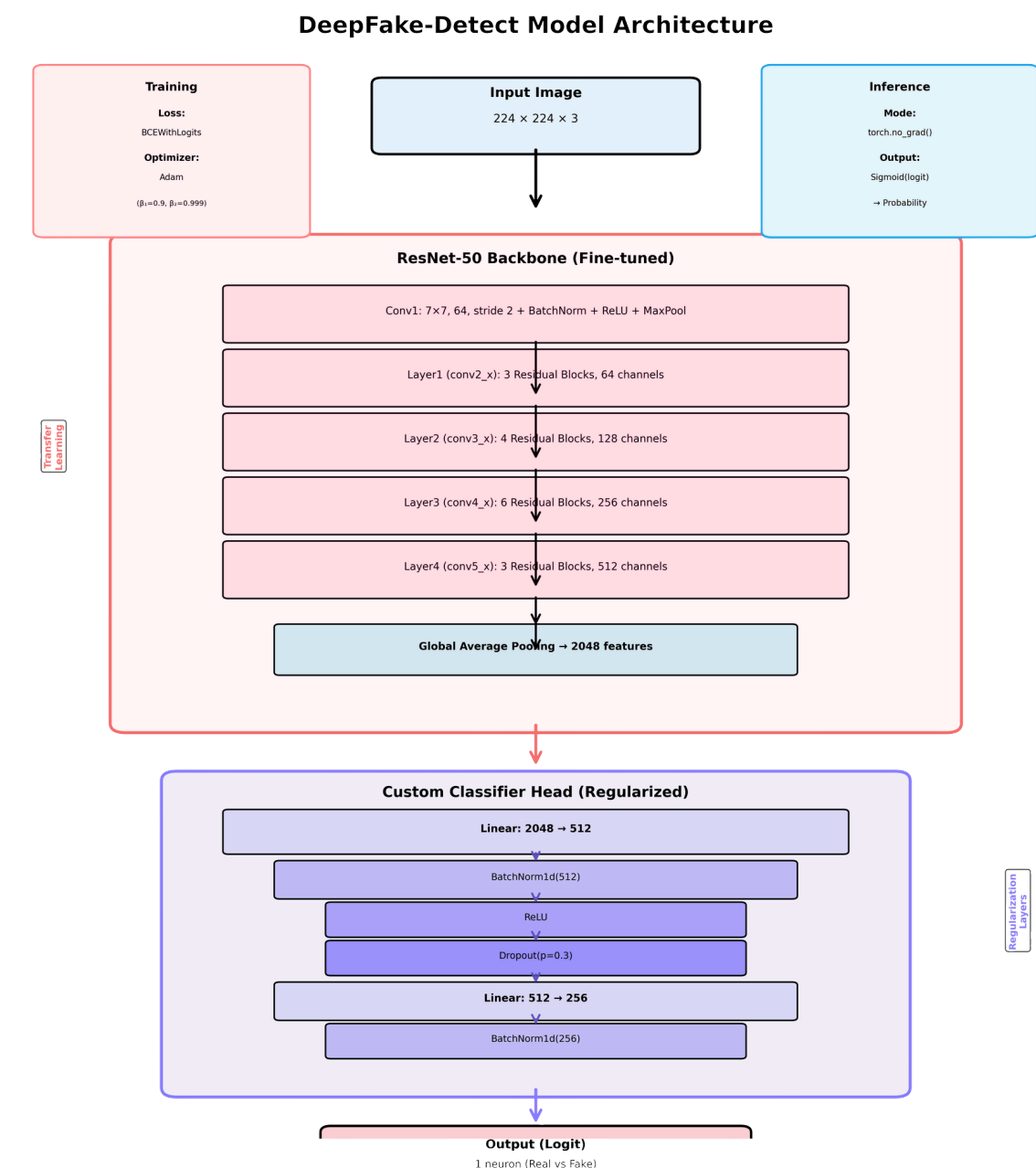
- Chosen for its excellent trade-off between representational capacity and computational cost.
- Residual connections help capture sophisticated artifacts

Custom Classifier Head:

- The original 1000-class head was replaced.
- Our head is a compact, regularized multi-layer perceptron:
- Linear(2048 → 512) → BN → ReLU → Dropout(0.3)
- Linear(512 → 256) → BN → ReLU → Dropout(0.2)
- Linear(256 → 1) (Logit Output)

Training Details:

- Loss: BCEWithLogitsLoss (for numerical stability).
- Optimizer: Adam (lr = 1e-4, weight decay = 1e-5).
- Batch Size: 32
- Hardware: NVIDIA RTX-class GPU, PyTorch, Mixed-Precision Training (AMP).



Results: quantitative Analysis

Backbone: ResNet-50

- Accuracy: 85.71%
- Macro F1-Score: 0.86

Confusion Matrix:

- True Negatives (Real): 23
- False Positives (Real -> Fake): 4
- True Positives (Fake): 19
- False Negatives (Fake -> Real): 3

Class	Precision	Recall	F1-score	Support
Fake	0.88	0.85	0.87	27
Real	0.83	0.86	0.84	22

Results: Ablation Study

(Proof Our Method Works)

Question Did our custom head and augmentations actually help?

Ablation Study Results (Table II):

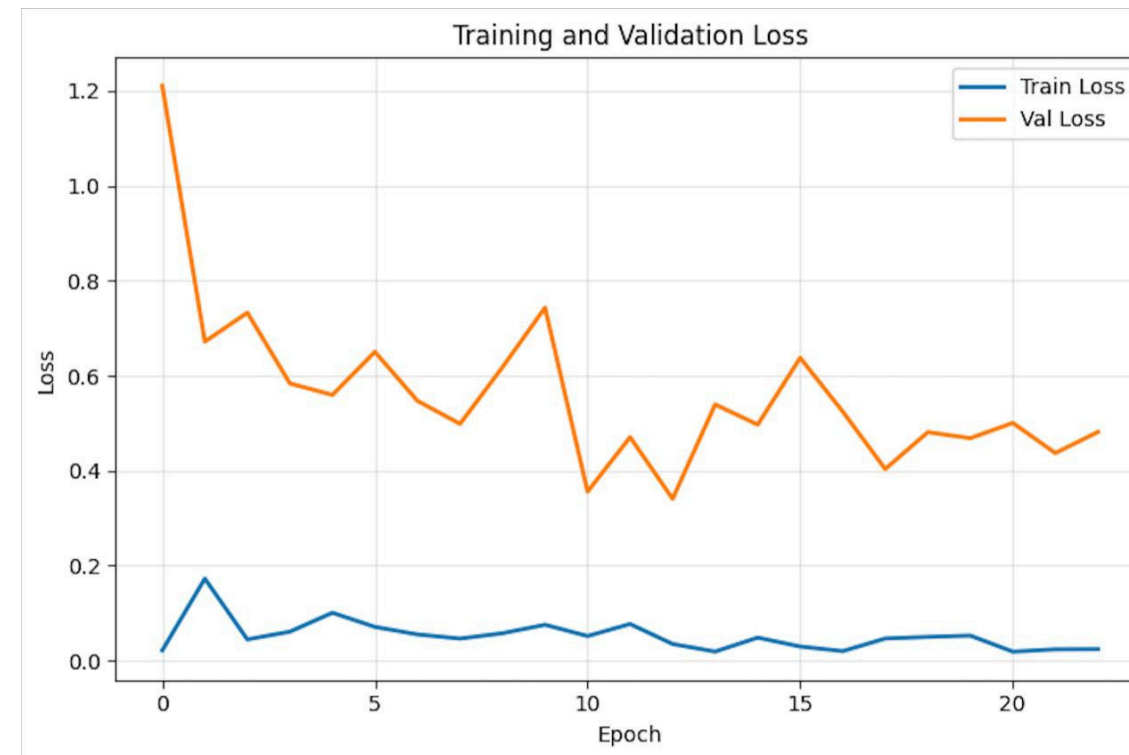
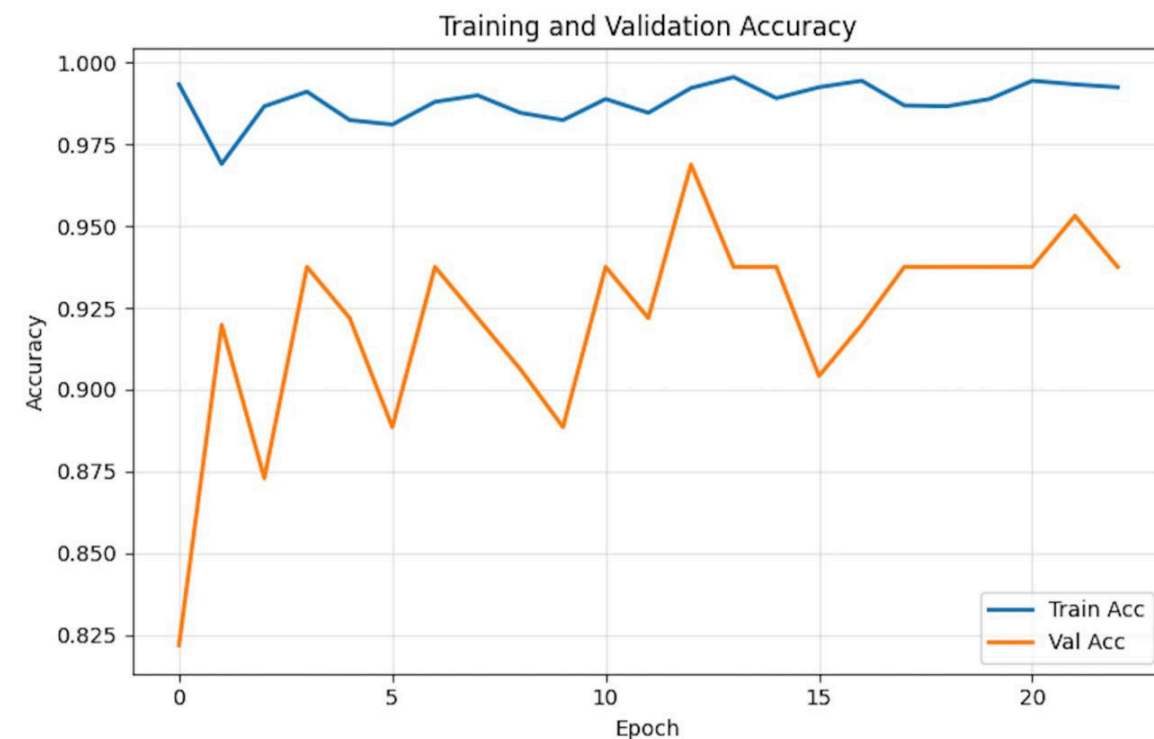
Configuration	Train Acc (%)	Test Acc (%)
ResNet-50 (Frozen) + linear head	72	71.43
ResNet-50 (Fine-tuned) + linear head	85	81.63
ResNet-50 + custom head (no augment)	96.5	78.2
Our Full Model (Custom Head + Augmentations)	93.5	85.71

Key Insight: The "no augment" model had 96.5% training accuracy but only 78.2% test accuracy—it was overfitting.

Conclusion: Our augmentations reduced training accuracy slightly but significantly improved test-set generalization, boosting it from 78.2% to 85.71%.

Results: Training Curves

Visuals:



Analysis:

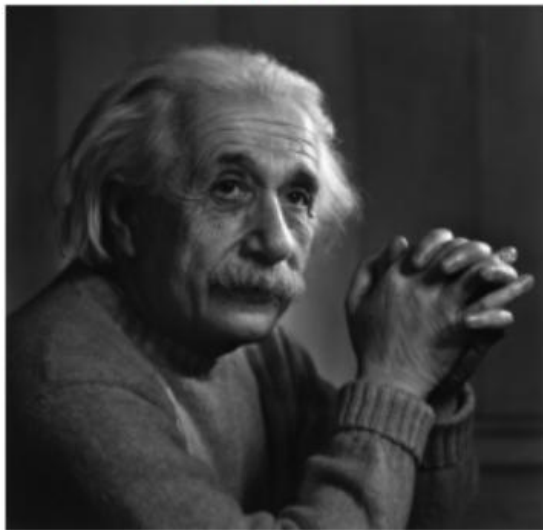
- The curves show steady convergence.
- Training accuracy is high (blue line, top chart) while validation accuracy (orange line) follows the trend.
- Crucially, the validation loss (orange line, bottom chart) does not diverge from the training loss (blue line).
- This demonstrates that our regularization (custom head + augmentations) was successful in preventing severe overfitting.

Results: Qualitative Analysis (Where it Fails)

Visuals

Success

Actual: real
Pred: real
Prob: 1.00



Actual: fake
Pred: fake
Prob: 0.00



Fail

Actual: real
Pred: fake
Prob: 0.00



Actual: fake
Pred: real
Prob: 0.95



Deployment: Server (FastAPI)

- **Technology:** Python, FastAPI, and Uvicorn workers.
- **Endpoint:** A clean /predict endpoint.
- **Workflow:**
 - Receives an uploaded image (multipart/form-data).
 - Preprocesses the image (resize, normalize).
 - Performs inference in torch.no_grad() mode.
 - Moves model and tensors to CUDA if a GPU is available.
 - Returns a clean JSON response with:
 - Predicted label ("Real" or "Fake")
 - Confidence score (sigmoid of the logit)
 - Inference time (in milliseconds)

Deployment: Client (React Native)

- **Technology:** Deployment: Client (React Native)
- **Purpose:** Consumes the FastAPI inference service
- **User Features:**
 - Image picker to select from camera or gallery.
 - Asynchronous upload with a progress indicator.
 - Clear results view with a color-coded label and percentage confidence.
 - Option to save history and export logs for forensic analysis

Limitations & Future Work

- **Current Limitations:**

1. **Image-Only:** The system is not designed for video, which requires temporal consistency checks.
2. **Dataset Scale:** The held-out test set of 49 images is small. A larger, more diverse benchmark is needed for stronger statistical claims.

- **Future Directions:**

1. **Frequency-Domain Features:** Incorporate DCT coefficients to find artifacts invisible in the spatial domain.
2. **Adversarial Training:** Actively train the model against perturbations designed to fool it.
3. **Explainability:** Add GRAD-CAM overlays to highlight which parts of the image the model thinks are fake.
4. **On-Device Inference:** Create a quantized model that can run directly on the phone for privacy and speed.

Conclusion

- **We Proved:** A fine-tuned ResNet-50 combined with a custom regularized head and targeted augmentations (JPEG, blur) improves robustness against realistic degradations.
- **We Presented:** DeepFake-Detect, a complete, end-to-end pipeline.
- **Our Result:** 85.71% accuracy and a deployed FastAPI + React Native application.
- **Acknowledgements:** We thank our course instructor, Dr. Animesh Chaturvedi, for his guidance and feedback.